

Plano de Execucao do Trabalho

Processador ARMv7 no Logisim com unidade de controle microprogramada (uROM mais PLAs)

Documento de planejamento e divisao de tarefas. Base tecnica: referencia ARMv7 destilada do manual ARM DDI 0406C.c e repositorio de partida ARMv7 Logisim (TU Darmstadt). Junho de 2026.

1. Visao geral do que vamos construir

O objetivo e projetar um subconjunto do processador ARMv7 no Logisim, com caminho de dados modular e, principalmente, uma unidade de controle **microprogramada** baseada em uma memoria de microcodigo (uROM) e em PLAs separadas. Esse controle microprogramado e o diferencial central do trabalho. Todas as instrucoes do ARM tem 32 bits e usam os bits 31 a 28 como codigo de condicao.

O caminho de dados envolve PC e logica de desvio, memoria de instrucoes e registro de instrucao, banco de 16 registradores de 32 bits, barrel shifter, geradores de imediato, ALU com flags NZCV, avaliador de condicao e memoria de dados. O controle precisa sequenciar fetch, decodificacao e execucao, despachar para a microrrotina certa de cada instrucao e anular efeitos quando a condicao falha.

O escopo segue tres fases: nucleo minimo (MOV, ADD, SUB, AND, ORR, CMP, B, branch condicional, LDR e STR com imediato), depois o restante do processamento de dados, deslocamentos e modos de endereco, e por fim multiplicacao, transferencia em bloco e branch com link.

2. O que ja temos no repositorio e o que da para aproveitar

O repositorio de partida e o **ARMv7 Logisim** (autoria de David Eckhardt, TU Darmstadt, 2021), um processador ARMv7 monociclo. Ele esta contido em um unico arquivo `Processor.circ` com varios subcircuitos, mais o README, a licenca e dois programas de teste em `examples`. O ponto mais importante: o caminho de dados ja esta pronto e e totalmente reaproveitavel, mas a unidade de controle dele e **cabeada** (montada com 62 portas logicas, sem nenhuma ROM ou PLA). Ou seja, o diferencial pedido, controle microprogramado, nao existe la e precisa ser construido do zero.

2.1 Inventario dos subcircuitos e status de aproveitamento

Bloco no repositorio	O que faz	Status para o nosso trabalho
RegisterField	Banco de registradores R0 a R14 mais a entrada do PC (modelo 15 mais 1)	Reaproveitar direto
ALU e ALUw/Flags	Soma, subtracao e logicas, com saidas de resultado e flags N, Z, C, V	Estender para o conjunto completo
Shifter	Barrel shifter com LSL, LSR, ASR e ROR	Estender (falta o RRX e o carry de saida do shifter)
Extender	Extensao de imediatos	Estender (ARMEExpandImm, ZeroExtend e SignExtend)
PCSrc	Selecao da proxima fonte do PC	

		Adaptar ao controle microprogramado
ROM de instrucoes e RAM de dados	Memoria de programa e memoria de dados	Reaproveitar direto
ControlUnit (cabeada)	Decodificador combinacional de sinais de controle	Substituir por uROM mais PLAs
main (topo)	Circuito de nivel superior que liga tudo	Adaptar a nova arquitetura de controle
examples (dois testes)	Programas em codigo de maquina ARM	Base para a bateria de testes

2.2 Instrucoes que o repositorio ja cobre e as que faltam

Pelo README, o projeto de partida ja executa `AND`, `SUB`, `ADD`, `TST`, `CMP`, `CMN`, `ORR`, `MOV`, `LSL`, `LSR`, `ASR`, `MVN`, `STR`, `LDR` e `B`, com execucao condicional e segundo operando deslocado. Ficam de fora e precisarao ser feitas: `EOR`, `RSB`, `ADC`, `SBC`, `RSC`, `TEQ`, `RRX`, `ROR`, `BIC`, todas as de multiplicacao e divisao, as cargas e armazenamentos especiais, o `BL` e as instrucoes diversas. Isso casa bem com o nosso faseamento: a fase 1 ja esta quase coberta no caminho de dados, e as fases 2 e 3 concentram o trabalho novo.

Resumo do aproveitamento. Reaproveitamos praticamente todo o caminho de dados (banco de registradores, ALU, shifter, extensor, memorias e logica de PC) e os programas de teste. Reconstruimos a unidade de controle como microcodigo em uROM mais PLAs, que e justamente a parte que vale o trabalho.

3. O material anexado cobre tudo? O que pesquisamos para preencher

O arquivo de referencia anexado e muito bom e cobre quase tudo do lado da arquitetura ARM: formato e decodificacao da instrucao, tabelas de opcode, flags afetadas, modos de endereco, pseudocodigo de apoio (`AddWithCarry`, deslocamentos, `ARMExpandImm`), programas de teste ja com o binario conferido e ate a estrutura do relatorio. Para documentar cada instrucao e montar o caminho de dados, o material e suficiente.

O que o material **nao** entrega na pratica, e que por isso pesquisamos:

3.1 Como construir o controle microprogramado dentro do Logisim

O anexo descreve o conceito (uROM, PLA de dispatch, PLA de condicao, microsequenciador) mas nao traz a receita pratica de montagem no Logisim, e o repositorio nao ajuda nisso porque seu controle e cabeado. A pesquisa confirmou um caminho concreto: o microsequenciador usa um registro de microendereço que alimenta a memoria de microcodigo; a cada ciclo, um multiplexador de proximo microendereço escolhe entre incrementar, saltar para um endereço absoluto vindo da propria microinstrucao, ou saltar para o endereço de entrada da instrucao gerado pela logica de mapeamento (a PLA de dispatch). Cada palavra da uROM carrega os sinais de controle do caminho de dados para aquele ciclo mais os bits de proximo microendereço.

Existe um exemplo completo e baixavel de CPU microcodificada feita no Logisim, com ROM de controle, ROM de decisao e um microassembler, alem de materiais de curso (NJIT e Georgia Tech) que mostram o esqueleto do microsequenciador. Esses servem de modelo direto para a nossa montagem. As referencias estao na secao 9.

3.2 O componente PLA do Logisim

Confirmamos que o Logisim Evolution tem um componente PLA nativo, com planos AND e OR programaveis, conteudo editavel de forma parecida com uma ROM e, importante para a entrega, com importacao e exportacao do conteudo por arquivo. Isso permite entregar a PLA de dispatch, a de condicao e a de funcao da ALU como arquivos separados, exatamente como o trabalho pede. A uROM tambem pode ser exportada como conteudo de ROM em arquivo.

Pendencia a confirmar com quem passou o trabalho. Nao temos no pacote o enunciado oficial da disciplina, apenas a referencia que o grupo destilou. Antes de comecar, conferir: prazo de entrega, conjunto minimo de instrucoes exigido, se o controle deve ser multiciclo, o formato exato dos arquivos de uROM e PLA esperados, e se a versao exigida e o Logisim classico ou o Logisim Evolution (o componente PLA nativo esta no Evolution).

4. Decisoese de arquitetura a fechar antes de comecar

Tres ou quatro decisoes definem o resto do projeto e devem ser combinadas por todos logo no inicio, para os blocos encaixarem.

1. **Semantica do PC.** Ao ler o PC, o ARM real entrega o endereco da instrucao mais 8. Decidir e justificar no relatorio se implementamos o mais 8 fiel ou se simplificamos para mais 4 ou mais 0 num modelo monociclo ou multiciclo.
2. **Monociclo dirigido por microcodigo (multiciclo).** O controle microprogramado e naturalmente multiciclo (fetch, decodificacao e ciclos de execucao). O caminho de dados do repositorio e monociclo. Decidir se mantemos os blocos e os dirigimos por um controlador microcodificado, acrescentando os latches necessarios (registro de instrucao e registros temporarios de operandos).
3. **Formato da microinstrucao.** Definir os campos e as larguras: proximo microendereco, bit de dispatch, selecao de leitura de registradores, operacao da ALU, fontes A e B, controle do shift, leitura e escrita de memoria, escrita em registrador, escrita de flags, fonte do PC e bit de fim. Escolher entre microcodigo horizontal (mais simples de ler, palavra mais larga) ou vertical (mais compacto, precisa decodificar).
4. **Tratamento da condicao.** Confirmar que a PLA de condicao recebe cond mais NZCV e decide se a instrucao escreve seus efeitos ou vira NOP, transformando o resultado da condicao em uma habilitacao global de escrita.

5. Faseamento do trabalho

Fase	Conteudo	Saída
Fase 0 Preparacao		

	Clonar o repositório, instalar o Logisim, fechar as decisões da seção 4, definir o formato da microinstrução e padronizar o repositório do grupo.	Convenções combinadas e ambiente pronto para todos.
Fase 1 Núcleo mínimo	MOV, ADD, SUB, AND, ORR, CMP, B, branch condicional, LDR e STR com imediato, já com o controle microprogramado funcionando de ponta a ponta.	Processador mínimo simulando os primeiros testes.
Fase 2 Ampliação	Restante do processamento de dados (ADC, SBC, RSB, RSC, EOR, BIC, MVN, TST, TEQ, CMN), deslocamentos (LSL, LSR, ASR, ROR, RRX), e os modos offset, indexação prévia e indexação posterior em carga e armazenamento, mais LDRB e STRB.	Conjunto amplo de instruções validado.
Fase 3 Extras e fechamento	MUL e MLA, transferência em bloco (LDM, STM, PUSH, POP), BL e BX, MOVW e MOVT, conforme o tempo. Integração final, bateria de testes completa e relatório.	Entrega final com circuito, arquivos de uROM e PLA, testes e relatório.

6. Divisão do trabalho entre os grupos

Quatro duplas, com cargas equilibradas e dependências pensadas para correr em paralelo. Cada dupla constrói seu bloco para o núcleo na fase 1, amplia na fase 2 e escreve a sua parte do relatório. A montagem e a integração no Logisim ficam concentradas na dupla responsável pelo circuito.

Grupo A | Vitor Oli e Renan

Caminho de dados aritmético e lógico

- ALU baseada em AddWithCarry mais bloco lógico (AND, OR, XOR, NOT), com saídas de resultado, carry e overflow, e a inversão de operando mais carry de entrada que unifica ADD, SUB, RSB, ADC, SBC e RSC numa única soma.
- Registrador de flags N, Z, C, V com habilitação governada pelo bit S e pelas comparações CMP, CMN, TST e TEQ.
- Barrel shifter completo (LSL, LSR, ASR, ROR e RRX) com a saída de carry do shifter, controlado por type e imm5 ou pelo byte baixo de Rs.
- Geradores de imediato: ARMEExpandImm para processamento de dados, ZeroExtend para carga e armazenamento e SignExtend para branch.

Grupo B | Thiago Paiva e Sofia

Montagem e integracao do circuito no Logisim

- Importar e adaptar os subcircuitos prontos do repositorio (banco de registradores, memorias de instrucao e de dados) para o projeto do grupo.
- Montar o PC, o somador, a logica de desvio (mux entre PC mais 4, alvo do branch e valor da ALU) e o registro de instrucao.
- Integracao de nivel superior: ligar caminho de dados, controle e memorias no circuito principal e garantir a simulacao ciclo a ciclo, com os valores de registradores visiveis no topo.
- Padronizar a organizacao do arquivo, com um subcircuito por bloco, e manter o circuito sempre montavel para os demais grupos testarem.

Grupo C | Thiago e Vitor Riguette

Unidade de controle microprogramada

- uROM com o microcodigo: definir o formato da microinstrucao e preencher as microrrotinas de fetch, decodificacao e execucao por classe de instrucao.
- Microsequenciador: microcontador, multiplexador de proximo microendereço (incrementar, salto absoluto e mapeamento por dispatch) e o registro de microendereço.
- PLA de dispatch: a partir dos campos do registro de instrucao, produzir o microendereço inicial de cada instrucao.
- Integrar o avaliador de condicao entregue pelo Grupo D na logica de commit, fazendo a instrucao virar NOP quando a condicao falha. PLA opcional de funcao da ALU, mapeando opcode para a operacao e os sinais de inversao e carry.
- Exportar a uROM e cada PLA como arquivos separados de conteudo, como a entrega exige.

Grupo D | Yan e Silvio

Avaliador de condicao, documentacao e testes

- Avaliador de condicao como subcircuito proprio: a partir do campo cond (bits 31 a 28) e das flags N, Z, C, V, gerar o sinal de condicao satisfeita, implementando diretamente a tabela de codigos de condicao da secao A8.3. Bloco pequeno e inteiramente definido pela tabela, entregue ao grupo de controle.
- Documentar a ficha de cada instrucao (mnemonica e sintaxe, diagrama de campos, opcode, flags afetadas, funcionamento e modo de endereco) e o modelo de programacao adotado, com a decisao sobre a semantica do PC.
- Montar e rodar a bateria de testes em assembly e binario, registrando resultado esperado versus observado no Logisim (registradores, memoria e flags).
- Carregar cada programa de teste na ROM do Logisim e registrar o estado final de registradores, memoria e flags, a partir do circuito integrado entregue pelo Grupo B.
- Montar o diagrama de blocos do caminho de dados para o relatorio, desenhado a partir do circuito ja pronto do Grupo B.
- Elaboracao do relatorio final com todas as entregas.

Responsabilidades compartilhadas por todos. Cada dupla escreve a secao do relatorio referente ao seu bloco, participa da validacao dos testes e ajuda na integracao final. As decisoes da secao 4 sao tomadas em conjunto na fase 0.

7. Cronograma sugerido e dependencias

Momento	Foco	Quem depende de quem
Fase 0	Decisoes de arquitetura, formato da microinstrucao e ambiente.	Todos juntos. Define os contratos de sinais entre os blocos.
Fase 1A	Cada dupla entrega seu bloco para o nucleo minimo. O Grupo D entrega cedo o avaliador de condicao.	Grupo B precisa dos blocos de A e C para integrar. Grupo C precisa do mapa de sinais que B vai consumir e do avaliador de condicao do Grupo D.
Fase 1B	Primeira integracao e testes do nucleo.	Grupo B integra. Grupo D carrega os testes, registra os estados e desenha o diagrama de blocos a partir do circuito integrado.
Fase 2A	Ampliacao do conjunto de instrucoes.	A amplia ALU e shifter, C amplia microcodigo e PLAs, B reintegra, D retesta.
Fase 2B	Conforme o tempo, congelamento e relatorio final.	Todos. D consolida o relatorio.

Ajustar a quantidade de semanas ao prazo real do enunciado.

8. Checklist de entregaveis

- Arquivo do circuito no Logisim com o caminho de dados completo e a unidade de controle microprogramada.
- Arquivo com o conteudo da uROM (microcodigo).
- Arquivos separados com o conteudo de cada PLA utilizada (dispatch, condicao e, se houver, funcao da ALU).
- Programas de teste em assembly e o respectivo binario, com resultado esperado e observado.
- Relatorio com introducao e objetivos, modelo de programacao, conjunto de instrucoes documentado, arquitetura do caminho de dados, unidade de controle microprogramada com as tabelas, testes e a tabela de responsabilidades.

9. Referencias

- Referencia ARMv7 destilada do manual ARM Architecture Reference Manual, ARMv7 A e R, ARM DDI 0406C.c (documento anexado ao projeto).
- Repositorio de partida, processador ARMv7 monociclo no Logisim: <https://github.com/TeflonAntihaft/ARMv7-Logisim>
- Exemplo completo de CPU microcodificada no Logisim, com ROM de controle, ROM de decisao e microassembler (Warren Toomey): <https://minnie.tuhs.org/Programs/UcodeCPU/index.html>

- Experimento de projeto de microsequenciador, NJIT: <https://web.njit.edu/~gilhc/ECE495/ece495-II.htm>
- Microsequenciador com ROM de estados, Georgia Tech CS2200: <https://faculty.cc.gatech.edu/~rama/CS2200-External/projects/p2/Bonus%20Microcontroller/microsequenced-rom.html>
- Logisim Evolution, ferramenta com componente PLA nativo: <https://github.com/logisim-evolution/logisim-evolution>